LAB MANUAL

Visual Programming

DEPARTMENT OF SOFTWARE ENGINEERING
FACULTY OF ENGINEERING & COMPUTING
NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD

Preface

This lab manual has been prepared to facilitate the students of software engineering in studying and implementing Desktop and Web based applications along with the Database connectivity. Initially, it covers basic UI controls, their properties, and related functions. Then, it revolves around the UI controls that support data to be shown in them and covers the concepts of data binding. Afterwards, it covers database connectivity and operations in .NET applications through ADO.NET and other supportive concepts. The later part of the lab talks about Web Services and APIs, and focuses on programming Web Services in .NET Core and connecting to different APIs and Web Services through client applications.

Tools/ Technologies

- Visual Studio, SQL Server Management Studio (SSMS)
- C#, SQL
- .NET, ASP.NET, ASP.NET Core

TABLE OF CONTENTS

LAB 1: Windows Form, Properties & Event Handlers

Objectives

- Learn basics of Windows Forms Applications
- Understand the use of Properties
- Design interactive Windows Forms with menus, buttons, and input fields.
- Implement Event Handlers

Theoretical Description

Windows Forms Applications

- Windows Forms is a part of the .NET framework used to create desktop applications with a graphical user interface (GUI).
- It allows developers to drag and drop components like buttons, text boxes, and menus to design an application visually.
- The underlying code links user actions to application functionality, making applications dynamic and interactive.

Properties

- Properties in C# provide a way to access and manipulate private fields of a class in a controlled manner.
- They allow you to enforce rules or validation when reading from or writing to a field.
- Example: A Balance property in an Account class can ensure that only valid values are assigned to the balance field.

Event Handlers

- Event handlers are methods that respond to specific actions, like button clicks or menu selections.
- In Windows Forms, event handlers are connected to GUI components. For example, clicking a button can trigger an event handler to calculate and display results.
- This interaction creates dynamic behavior in the application.

Lab Task

Create a Windows Forms application with a calculator and an account details form.

- Implement a parent `Calculator` class for basic operations (Add, Subtract) and a child `ScientificCalculator` class for advanced features (Square Root, Power).

- Design a form with menus, buttons, and text boxes for performing calculations and displaying results.
- Create an `Account` class with private fields, public properties, and a form to accept and display account details. Ensure buttons trigger appropriate event handlers to process user inputs and display results.

Solution

```
using System;
using System.Windows.Forms;

namespace VisualProgrammingLab
{
    // Part 1: Parent and Child Classes for Calculator
    public class Calculator
    {
        public double Add(double a, double b) => a + b;
        public double Subtract(double a, double b) => a - b;
    }

    public class ScientificCalculator : Calculator
    {
        public double SquareRoot(double a) => Math.Sqrt(a);
        public double Power(double a, double b) => Math.Pow(a, b);
    }

    // Part 2: Account Class with Properties
    public class Account
    {
        private string accountName;
        private double balance;

        public string AccountName
        {
            get { return accountName; }
            set { accountName = value; }
        }

        public double Balance
        {
            get { return balance; }
            set { balance = value; }
        }
    }

    // Part 3: Main Form Design
    public partial class MainForm : Form
    {
        public MainForm()
        {
            InitializeComponent();
        }

        // Event handler for Calculator operations
```

```
        private void btnCalculate_Click(object sender, EventArgs e)
        {
            ScientificCalculator calc = new ScientificCalculator();
            double num1 = double.Parse(txtNumber1.Text);
            double num2 = double.Parse(txtNumber2.Text);

            // Perform addition (or any other operation)
            double result = calc.Add(num1, num2);
            txtResult.Text = result.ToString();
        }

        // Event handler for Account operations
        private void btnSaveAccount_Click(object sender, EventArgs e)
        {
            Account account = new Account
            {
                AccountName = txtAccountName.Text,
                Balance = double.Parse(txtBalance.Text)
            };

            // Display account details
            MessageBox.Show($"Account Saved!\nName:
{account.AccountName}\nBalance: {account.Balance}");
        }
    }
}
```

LAB 2: Database Connectivity in with ADO.NET

Objectives

- Understand how to connect a Windows Forms application to SQL Server using ADO.NET
- Learn to perform CRUD operations through ADO.Net
- Display data in a DataGridView control.

Theoretical Description

ADO.NET

- ADO.NET is a set of classes in .NET Framework that provide data access services. It allows applications to connect to databases, retrieve data, and manipulate it.
- The primary components for database interaction are:
- SqlConnection: Represents the connection to a SQL Server database.
- SqlCommand: Executes queries or commands against the SQL Server database.
- SqlDataAdapter: Acts as a bridge between the dataset and the SQL Server database. It retrieves data and updates the database.
- SqlDataReader: Used to fetch data from the database in a forward-only, read-only mode.
- ExecuteNonQuery: Executes commands that do not return results, such as INSERT, UPDATE, and DELETE.
- Adapter.Fill: Fills a DataSet or DataTable with data from the database, which can then be bound to controls like DataGridView.

DataGridView

- The DataGridView is a Windows Forms control used to display tabular data. You can bind it to a DataTable or DataSet to show database records.
- After performing operations like Insert, Update, and Delete, the DataGridView can be refreshed to reflect the changes.

Lab Task

Create a Windows Forms application that connects to a SQL Server database and performs Insert, Update, and Delete operations using ADO.NET.

- Use a SqlConnection to establish a connection to SQL Server.
- Use a SqlDataAdapter and DataTable to fetch data from the database and display it in a DataGridView.
- Implement buttons for Search, Insert, Update, and Delete operations. After each operation, refresh the DataGridView to show the updated data.

Solution

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace ADO_NET_Lab
{
    public partial class MainForm : Form
    {
        // Connection string to SQL Server (replace with your actual server and
database details)
        private string connectionString = "Server=localhost;
Database=YourDatabase; Integrated Security=True;";

        public MainForm()
        {
            InitializeComponent();
        }

        // Method to fetch data from the database and bind it to DataGridView
        private void LoadData()
        {
            using (SqlConnection connection = new SqlConnection(connectionString))
            {
                string query = "SELECT * FROM YourTable"; // Replace 'YourTable'
with your actual table name
                SqlDataAdapter dataAdapter = new SqlDataAdapter(query,
connection);
                DataTable dataTable = new DataTable();
                dataAdapter.Fill(dataTable);
                dataGridView1.DataSource = dataTable; // Bind data to DataGridView
            }
        }

        // Insert data into the database
        private void btnInsert_Click(object sender, EventArgs e)
        {
            using (SqlConnection connection = new SqlConnection(connectionString))
            {
                string query = "INSERT INTO YourTable (Column1, Column2) VALUES
(@value1, @value2)";
                SqlCommand command = new SqlCommand(query, connection);
                command.Parameters.AddWithValue("@value1", txtValue1.Text); //
Assuming txtValue1 is the input for Column1
                command.Parameters.AddWithValue("@value2", txtValue2.Text); //
Assuming txtValue2 is the input for Column2

                connection.Open();
                command.ExecuteNonQuery(); // Execute the INSERT command
                connection.Close();
            }

            LoadData(); // Refresh the DataGridView after insertion
        }

        // Update data in the database
        private void btnUpdate_Click(object sender, EventArgs e)
```

```
        {
            using (SqlConnection connection = new SqlConnection(connectionString))
            {
                string query = "UPDATE YourTable SET Column1 = @value1 WHERE
Column2 = @value2";
                SqlCommand command = new SqlCommand(query, connection);
                command.Parameters.AddWithValue("@value1", txtValue1.Text); //
Assuming txtValue1 is the updated value for Column1
                command.Parameters.AddWithValue("@value2", txtValue2.Text); //
Assuming txtValue2 is used for identifying the record to update

                connection.Open();
                command.ExecuteNonQuery(); // Execute the UPDATE command
                connection.Close();
            }

            LoadData(); // Refresh the DataGridView after update
        }

        // Delete data from the database
        private void btnDelete_Click(object sender, EventArgs e)
        {
            using (SqlConnection connection = new SqlConnection(connectionString))
            {
                string query = "DELETE FROM YourTable WHERE Column2 = @value2";
                SqlCommand command = new SqlCommand(query, connection);
                command.Parameters.AddWithValue("@value2", txtValue2.Text); //
Assuming txtValue2 identifies the record to delete

                connection.Open();
                command.ExecuteNonQuery(); // Execute the DELETE command
                connection.Close();
            }

            LoadData(); // Refresh the DataGridView after deletion
        }

        // Load data when the form loads
        private void MainForm_Load(object sender, EventArgs e)
        {
            LoadData(); // Fetch and display data in the DataGridView
        }
    }
}
```

LAB 3: Centralized Database Operations Using Connection-less Approach

Objectives

- Learn to design a database in SQL Server for a Windows Forms application.
- Implement a centralized DbConn class to handle database operations.
- Understand and use a connection-less approach to interact with the database.
- Use ADO.NET components such as DataSet, DataTable, and DataAdapter for retrieving and displaying data.

Theoretical Description

Centralized Database Operations with DbConn Class

- The DbConn class acts as a centralized helper class to manage database connections and operations. This class handles opening and closing connections, executing SQL commands, and managing exceptions.
- The goal of centralizing database operations in a class is to avoid repetition and make the code easier to maintain.

Connection-less Approach in ADO.NET

- A connection-less approach means that you can retrieve data from the database without keeping an open connection throughout the process.
- ADO.NET achieves this using DataSet and DataTable, which are disconnected objects. Once data is fetched using a SqlDataAdapter, it is stored in a DataTable or DataSet object, which can be used without an active connection.
- DataAdapter.Fill: Fills a DataTable or DataSet with data from the database.
- The DataSet can hold multiple DataTable objects and supports in-memory data management, allowing you to work with the data even when the connection is closed.

Using DataSet, DataTable, and DataAdapter

- DataSet: A collection of DataTable objects. It represents the entire data structure and supports multiple tables and relationships between them.
- DataTable: Represents a single table of data, and you can access and manipulate data row by row.
- DataAdapter: Acts as a bridge to fetch data from the database and load it into a DataSet or DataTable.

Lab Task

Create a Windows Forms application that connects to a SQL Server database using a centralized `DbConn` class and performs a search operation using a connection-less approach.

- Design a database in SQL Server with an `Employees` table.
- Implement a `DbConn` class to handle database connectivity and operations like searching records.
- Use `DataSet` and `DataTable` to fetch and display records in a `DataGridView`.
- Perform a search operation by employee name, and show the results in the `DataGridView`.

Solution

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace ADO_NET_Lab
{
    public partial class MainForm : Form
    {
        // Centralized DbConn class for handling database operations
        public class DbConn
        {
            private string connectionString = "Server=localhost;
Database=YourDatabase; Integrated Security=True;";

            // Method to fetch data from the database using a connection-less
approach
            public DataTable GetEmployeeRecords(string searchQuery)
            {
                SqlConnection connection = new SqlConnection(connectionString);
                SqlDataAdapter dataAdapter = new SqlDataAdapter("SELECT * FROM
Employees WHERE FirstName LIKE @searchQuery", connection);
                dataAdapter.SelectCommand.Parameters.AddWithValue("@searchQuery",
"%" + searchQuery + "%");

                DataTable dataTable = new DataTable();
                connection.Open();
                dataAdapter.Fill(dataTable); // Fill DataTable with the data from
database
                connection.Close();
                return dataTable;
            }
        }

        public MainForm()
        {
            InitializeComponent();
        }

        // Button click event to search employee records
```

```
        private void btnSearch_Click(object sender, EventArgs e)
        {
            string searchQuery = txtSearch.Text; // Assuming txtSearch is a
TextBox for employee name input

            DbConn dbConn = new DbConn();
            DataTable results = dbConn.GetEmployeeRecords(searchQuery); // Fetch
records using DbConn class

            dataGridView1.DataSource = results; // Bind data to DataGridView
        }

        // Load data when the form loads (Optional - for initial data load)
        private void MainForm_Load(object sender, EventArgs e)
        {
            DbConn dbConn = new DbConn();
            DataTable initialData = dbConn.GetEmployeeRecords(""); // Fetch all
records
            dataGridView1.DataSource = initialData; // Bind data to DataGridView
        }
    }
}
```

LAB 4: ASP.Net Basics & Master Pages

Objectives

- Learn the basics of Web Applications and ASP.NET Life Cycle.
- Understand the concept and usage of Master Pages in ASP.NET.
- Implement AJAX functionality to enhance user interaction without full page reloads.
- Develop multiple pages that use the same or different Master Pages.

Theoretical Description

Introduction to ASP.NET Web Applications

- ASP.NET is a web development framework used for building dynamic web applications. Unlike traditional static HTML pages, ASP.NET Web Forms allow you to develop interactive and data-driven web pages.
- A Web Application in ASP.NET consists of various components, including Web Forms, Master Pages, and Web Services.

ASP.NET Life Cycle

- The ASP.NET Life Cycle is a sequence of events that occurs during the processing of a web request. It begins with the Request being sent to the server and ends with the Response being sent back to the client.
- Key stages include:
    - **Page Request**
    - **Page Initialization**
    - **Page Load**
    - **Postback Data Handling**
    - **Page Rendering**
    - **Unload**

Master Pages in ASP.NET

- Master Pages in ASP.NET provide a template for creating consistent layouts across multiple pages in a web application. They are used to define a common structure (e.g., header, footer, navigation) for the pages.
- Content pages can refer to a Master Page, which allows content pages to inherit common layouts while enabling dynamic content to be displayed.

AJAX in ASP.NET

- AJAX (Asynchronous JavaScript and XML) allows web pages to request data from the server and update parts of the page without refreshing the entire page. This improves the user experience by making web applications faster and more interactive.

- In ASP.NET, AJAX is implemented using the `ScriptManager` and `UpdatePanel` controls. The `ScriptManager` allows you to enable AJAX functionality on the page, and `UpdatePanel` controls allow partial page updates.

Lab Task

- Create an ASP.NET Web Application project in Visual Studio.
- Design and implement a Master Page that includes a common header, footer, and navigation menu.
- Create multiple content pages that use the Master Page for a consistent layout.
- Implement AJAX functionality using ScriptManager and UpdatePanel to display dynamic content on a page without a full reload.
- Test the pages under different Master Pages and ensure that AJAX works as expected.

Solution

```
// Master Page Code - MasterPage.master

<%@ Master Language="C#" AutoEventWireup="true" CodeFile="MasterPage.master.cs"
Inherits="MasterPage" %>
<!DOCTYPE html>
<html>
<head runat="server">
    <title>ASP.NET Master Page Example</title>
    <asp:ScriptManager ID="ScriptManager1" runat="server" />
</head>
<body>
    <div id="header">
        <h1>Welcome to My Website</h1>
        <ul>
            <li><a href="Default.aspx">Home</a></li>
            <li><a href="About.aspx">About</a></li>
            <li><a href="Contact.aspx">Contact</a></li>
        </ul>
    </div>

    <div id="content">
        <asp:ContentPlaceHolder ID="ContentPlaceHolder1" runat="server" />
    </div>

    <div id="footer">
        <p>© 2025 MyWebsite</p>
    </div>
</body>
</html>

// Content Page Code - Default.aspx
```

```
<%@ Page Language="C#" MasterPageFile="~/MasterPage.master" AutoEventWireup="true"
CodeFile="Default.aspx.cs" Inherits="Default" %>

<asp:Content ID="Content1" ContentPlaceHolderID="ContentPlaceHolder1"
runat="server">
    <h2>Home Page</h2>
    <p>Welcome to our home page. Below is an AJAX-enabled section:</p>

    <asp:UpdatePanel ID="UpdatePanel1" runat="server">
        <ContentTemplate>
            <asp:Button ID="btnLoadData" runat="server" Text="Load Data"
OnClick="btnLoadData_Click" />
            <asp:Label ID="lblData" runat="server" Text="Data will appear
here."></asp:Label>
        </ContentTemplate>
    </asp:UpdatePanel>
</asp:Content>

// Content Page Code - About.aspx

<%@ Page Language="C#" MasterPageFile="~/MasterPage.master" AutoEventWireup="true"
CodeFile="About.aspx.cs" Inherits="About" %>

<asp:Content ID="Content2" ContentPlaceHolderID="ContentPlaceHolder1"
runat="server">
    <h2>About Us</h2>
    <p>This is the About Us page, which shares information about the website.</p>
</asp:Content>

// Code-behind for Default.aspx - Default.aspx.cs

using System;

public partial class Default : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {
    }

    protected void btnLoadData_Click(object sender, EventArgs e)
    {
        // Simulating fetching data and updating the Label dynamically
        lblData.Text = "Data loaded successfully at " + DateTime.Now.ToString();
    }
}
```

LAB 5: Implementing Three-Tier Architecture

Objectives

- Learn the concept and use of Dynamic Link Libraries (DLLs) in .NET.
- Understand how to build and reference Class Libraries in a project.
- Implement a Three-Tier Architecture (Presentation Layer, Business Layer, and Data Access Layer).
- Use Properties to manage data flow between different layers.

Theoretical Description

Dynamic Link Libraries (DLLs)

- A Dynamic Link Library (DLL) is a collection of reusable code, functions, and resources that can be accessed by multiple applications. It is a file that contains compiled code that can be loaded dynamically by applications at runtime.
- DLLs help in reducing memory usage and making programs more modular and reusable. In .NET, DLLs are used to encapsulate functionality in separate units that can be easily referenced by other projects.

Class Libraries in .NET

- A Class Library is a project that contains reusable classes, methods, and properties which can be referenced by other projects. Building a class library allows the developer to create a central location for common functionality that can be shared across multiple applications.
- Class libraries are compiled into DLLs and referenced in other projects. By separating functionality into distinct libraries, the code becomes more maintainable and organized.

Three-Tier Architecture

- Three-tier architecture is a well-known software architecture pattern that separates an application into three layers:
    1. **Presentation Layer** (UI): The topmost layer responsible for displaying data and receiving user input. It interacts with the Business Layer to request and display information.
    2. **Business Layer**: The middle layer that contains the application's logic. It processes input from the Presentation Layer, performs necessary operations, and communicates with the Data Access Layer for data retrieval or storage.
    3. **Data Access Layer**: The bottom layer responsible for interacting with the database. It contains classes that perform operations like fetching, updating, or deleting data from a database.

Using Properties in Three-Tier Architecture

- Properties are used to manage data across different layers. They allow data to be passed between layers without directly exposing the underlying fields, enabling a clean separation of concerns and encapsulation.

Lab Task

- Create a Class Library project that contains a Business Layer class with properties and methods for performing operations (e.g., CalculateTotal or GetCustomerData).
- Build a Data Access Layer in another class library that includes methods to retrieve data from a database (e.g., using ADO.NET).
- Develop a Presentation Layer that interacts with the Business Layer to display data to the and handles user input.
- Reference the Class Libraries (Business Layer and Data Access Layer) in your Presentation Layer.
- Implement a simple functionality like calculating totals or fetching customer data, demonstrating the use of the three layers working together.

Solution

```
// Data Access Layer - DataAccessLayer.cs (Class Library)
using System;
using System.Data.SqlClient;

namespace DataAccessLayer
{
    public class DataAccess
    {
        private string connectionString = "your_connection_string_here";

        public string GetCustomerData(int customerId)
        {
            string customerName = string.Empty;
            using (SqlConnection connection = new SqlConnection(connectionString))
            {
                string query = "SELECT Name FROM Customers WHERE CustomerID = 
@CustomerID";
                SqlCommand command = new SqlCommand(query, connection);
```

```
                command.Parameters.AddWithValue("@CustomerID", customerId);
                connection.Open();
                customerName = command.ExecuteScalar()?.ToString();
            }
            return customerName;
        }
    }
}

// Business Layer - BusinessLayer.cs (Class Library)
using System;
using DataAccessLayer;

namespace BusinessLayer
{
    public class CustomerBusiness
    {
        private DataAccess dataAccess = new DataAccess();

        public string GetCustomerName(int customerId)
        {
            return dataAccess.GetCustomerData(customerId);
        }
    }
}

// Presentation Layer - Form1.cs (Windows Forms Application)
using System;
using System.Windows.Forms;
using BusinessLayer;

namespace PresentationLayer
```

```
{
    public partial class Form1 : Form
    {
        private CustomerBusiness customerBusiness = new CustomerBusiness();

        public Form1()
        {
            InitializeComponent();
        }

        private void btnGetCustomer_Click(object sender, EventArgs e)
        {
            int customerId = int.Parse(txtCustomerId.Text);
            string customerName = customerBusiness.GetCustomerName(customerId);
            lblCustomerName.Text = customerName;
        }
    }
}
```

LAB 6: Stored Procedures

Objectives

- Learn how to create stored procedures in SQL Server.
- Understand the use of stored procedures to perform database operations.
- Execute stored procedures from a .NET Windows Forms application.
- Write code behind the insert button to call the stored procedure and display results.

Theoretical Description

Stored Procedures in SQL Server

- A stored procedure is a set of SQL statements that can be executed as a single unit. Stored procedures allow the user to encapsulate logic for data manipulation (Insert, Update, Delete) or complex queries in the database itself.
- Advantages of stored procedures include improved performance, enhanced security (as SQL code is not exposed to the user), and the ability to reuse common logic across applications.

Creating a Stored Procedure

- In SQL Server, stored procedures are created using the `CREATE PROCEDURE` statement. Once created, they can be executed using the `EXEC` or `EXECUTE` keyword.
- Parameters can be passed to stored procedures to make them more flexible. These parameters can be input, output, or both.

Executing Stored Procedures in SQL Server

- Stored procedures are executed with the EXEC or EXECUTE command. You can also pass parameters to the stored procedure when executing it to retrieve data or perform operations.

Calling Stored Procedures from .NET Code

- In a .NET Windows Forms application, you can use SqlCommand objects to execute stored procedures. You can call the ExecuteNonQuery method to run procedures that perform actions (Insert, Update, Delete), or ExecuteReader to retrieve data.

Lab Task

- Create a stored procedure in SQL Server that inserts a record into a table. For example, a procedure to insert a new customer into a Customers table.
- Execute the stored procedure in SQL Server to ensure it works and performs the intended operation.
- Create a Windows Forms application with a Button to call the stored procedure when clicked. The form should take input from the user (such as customer name and email) and call the stored procedure to insert the data into the database.

Solution

1. **SQL Server: Creating the Stored Procedure**

```
-- Creating a stored procedure to insert customer data
CREATE PROCEDURE InsertCustomer
    @CustomerName NVARCHAR(100),
    @CustomerEmail NVARCHAR(100)
AS
BEGIN
    INSERT INTO Customers (Name, Email)
    VALUES (@CustomerName, @CustomerEmail)
END
```

2. **SQL Server: Executing the Stored Procedure**

To execute the stored procedure manually from SQL Server:

```
-- Execute stored procedure with parameters
EXEC InsertCustomer @CustomerName = 'John Doe', @CustomerEmail =
'johndoe@example.com';
```

3. **Windows Forms Application: Calling the Stored Procedure from Code**

```
// Form1.cs (Windows Forms Application)
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace StoredProcedureExample
{
    public partial class Form1 : Form
    {
        // Connection string to connect to the SQL Server database
        private string connectionString = @"connection_string ";

        public Form1()
        {
            InitializeComponent();
        }

        private void btnInsert_Click(object sender, EventArgs e)
        {
```

```
            // Getting data from the TextBoxes
            string customerName = txtCustomerName.Text;
            string customerEmail = txtCustomerEmail.Text;

            // Call the stored procedure to insert data
            InsertCustomer(customerName, customerEmail);
        }

        private void InsertCustomer(string customerName, string customerEmail)
        {
            // Create a new SqlConnection object to connect to the database
            using (SqlConnection connection = new SqlConnection(connectionString))
            {
                // Create a SqlCommand to call the stored procedure
                SqlCommand cmd = new SqlCommand("InsertCustomer", connection);
                cmd.CommandType = CommandType.StoredProcedure;

                // Add parameters to the SqlCommand
                cmd.Parameters.AddWithValue("@CustomerName", customerName);
                cmd.Parameters.AddWithValue("@CustomerEmail", customerEmail);

                try
                {
                    // Open the connection and execute the stored procedure
                    connection.Open();
                    cmd.ExecuteNonQuery();
                    MessageBox.Show("Customer inserted successfully!");
                }
                catch (Exception ex)
                {
                    MessageBox.Show("Error: " + ex.Message);
                }
            }
        }
    }
}
```

LAB 7: RDLC Reports

Objectives

- Learn how to create and design an RDLC report in Visual Studio.
- Understand how to add a table to an RDLC file and select data columns.
- Design a report header and footer.
- Write code to display a report when a button is clicked.
- Implement functionality to print the report.

Theoretical Description

RDLC (Report Definition Language Client-Side) Files

- RDLC files are used to design reports that can be generated client-side in a Windows Forms or Web application. They are flexible and can be used to display data in various formats such as tables, charts, and more.
- RDLC reports can be bound to data sources, such as DataTables, BindingLists, or business objects.

Creating and Designing RDLC Reports

- You can create an RDLC file by selecting "Report" from the project menu. This file allows you to design the layout and appearance of the report, including headers, footers, and data fields.
- RDLC files use data binding to link the data with the report. This allows dynamic population of fields such as employee names, job titles, and other details.

Report Header and Footer

- The report header contains information like the report title, company name, or other metadata.
- The footer typically contains information like the page number, report generation date, or contact details.

Code Behind to Display and Print the Report

- In the code behind (form's event handlers), you can bind the RDLC report to a data source and display it in a `ReportViewer` control.
- The `ReportViewer` control has a print option that can be used to print the report.

Lab Task

Create an RDLC file, add table and select columns, design header and footer of report. Write code behind view report button to display list of all employees and print it.

Solution

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;
using Microsoft.Reporting.WinForms;

namespace RDLCReportExample
{
    public partial class Form1 : Form
    {
        // Connection string to connect to the database
        private string connectionString = "your_connection_string_here";

        public Form1()
        {
            InitializeComponent();
        }

        private void btnViewReport_Click(object sender, EventArgs e)
        {
            // Fetch employee data
            DataTable employeeData = GetEmployeeData();

            // Bind data to the ReportViewer
            ReportDataSource rds = new ReportDataSource("EmployeeDataSet",
employeeData);
            reportViewer1.LocalReport.DataSources.Clear();
            reportViewer1.LocalReport.DataSources.Add(rds);
            reportViewer1.LocalReport.ReportPath = "EmployeeReport.rdlc"; // Path
to the RDLC file

            // Refresh the report to display it
            reportViewer1.RefreshReport();
        }

        private DataTable GetEmployeeData()
        {
            DataTable dt = new DataTable();
            using (SqlConnection connection = new SqlConnection(connectionString))
            {
                string query = "SELECT EmployeeID, EmployeeName, Position FROM
Employees";
                SqlDataAdapter da = new SqlDataAdapter(query, connection);
                da.Fill(dt);
            }
            return dt;
        }

        private void btnPrintReport_Click(object sender, EventArgs e)
        {
```

```
            // Print the report
            reportViewer1.PrintDialog();
        }
    }
}
```

LAB 8: Implementing Shopping Cart in ASP.NET

Objectives

- Learn how to implement a shopping cart system in ASP.NET.
- Understand the use of connection-oriented approach for data handling in web applications.
- Implement functionality to Add to Cart, View Cart, and Delete Items.
- Use session state to manage the shopping cart items.

Theoretical Description

Connection-Oriented Approach

- In the context of web development, a **connection-oriented approach** typically refers to establishing a session with the database for operations such as adding, updating, and deleting records. In ASP.NET, it is common to interact with a database for product data but maintain the shopping cart using **session state**.
- **Session** allows you to store user-specific data between requests, which makes it ideal for maintaining a shopping cart. Unlike connection-less approaches where data is directly retrieved for each request, connection-oriented approaches often use **SQL connections** to retrieve product details and session storage for cart items.

Shopping Cart

- A **shopping cart** is a critical feature of e-commerce websites where users can add products, view the cart contents, and remove items before proceeding to checkout.
- The shopping cart can be managed using **Session Variables** in ASP.NET, where each item added to the cart is stored as an object or dictionary. Each time the user interacts with the cart, the data is updated in the session.

Session State

- ASP.NET provides **Session State** to store values between HTTP requests. This is useful for storing user-specific data, such as shopping cart items.
- Each user has a unique session, and the session data is available across multiple pages during their browsing session.

Lab Task

- Create an ASP.NET web application and set up the necessary pages (e.g., Home.aspx, Cart.aspx).

- Design the product listing page where users can click an "Add to Cart" button to add products to their shopping cart.
- Store cart data in session: When a user adds an item to the cart, store the item in the session state as a dictionary or list.
- Create a Cart page: Display all items added to the cart along with quantity and price.
- Add functionality to delete items: Allow users to remove items from the cart on the Cart.aspx page.
- Ensure proper flow: From the product page, users can add products to the cart, view their cart, and delete items if needed.

Solution

1. **Create Product Page (Home.aspx)**
    - On this page, display a list of products and allow users to add items to the cart.

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Home.aspx.cs"
Inherits="ShoppingCartApp.Home" %>

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Product Listing</title>
</head>
<body>
    <form id="form1" runat="server">
        <h2>Products</h2>
        <asp:Button ID="btnAddToCart" runat="server" Text="Add to Cart"
OnClick="btnAddToCart_Click" />
        <asp:GridView ID="gvProducts" runat="server">
            <Columns>
                <asp:BoundField DataField="ProductID" HeaderText="ID"
SortExpression="ProductID" />
                <asp:BoundField DataField="ProductName" HeaderText="Product
Name" SortExpression="ProductName" />
                <asp:BoundField DataField="Price" HeaderText="Price"
SortExpression="Price" />
                <asp:TemplateField>
                    <ItemTemplate>
                        <asp:Button ID="btnAdd" runat="server" Text="Add to
Cart" CommandArgument='<%# Eval("ProductID") %>' OnClick="btnAdd_Click" />
                    </ItemTemplate>
                </asp:TemplateField>
            </Columns>
        </asp:GridView>
    </form>
</body>
</html>
```

Code-behind (Home.aspx.cs):

```
using System;
using System.Collections.Generic;

namespace ShoppingCartApp
```

```
{
    public partial class Home : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (!IsPostBack)
            {
                // Load products (for simplicity, using a static list)
                var products = new List<Product>
                {
                    new Product { ProductID = 1, ProductName = "Product 1",
Price = 100 },
                    new Product { ProductID = 2, ProductName = "Product 2",
Price = 200 },
                    new Product { ProductID = 3, ProductName = "Product 3",
Price = 300 }
                };
                gvProducts.DataSource = products;
                gvProducts.DataBind();
            }
        }

        protected void btnAdd_Click(object sender, EventArgs e)
        {
            Button btnAdd = (Button)sender;
            int productId = Convert.ToInt32(btnAdd.CommandArgument);
            // Add product to session cart
            AddToCart(productId);
        }

        private void AddToCart(int productId)
        {
            // Simulate a product (this would come from a database or an
API in a real application)
            var product = new Product { ProductID = productId, ProductName
= "Product " + productId, Price = 100 * productId };

            // Create cart session if it doesn't exist
            if (Session["Cart"] == null)
            {
                Session["Cart"] = new List<Product>();
            }

            List<Product> cart = (List<Product>)Session["Cart"];
            cart.Add(product);

            // Save back to session
            Session["Cart"] = cart;
        }
    }

    public class Product
    {
        public int ProductID { get; set; }
        public string ProductName { get; set; }
        public decimal Price { get; set; }
    }
}
```

2. **Create Cart Page (Cart.aspx)**:
    - On this page, display all items in the cart, allow users to delete items, and show total price.

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Cart.aspx.cs"
Inherits="ShoppingCartApp.Cart" %>

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Shopping Cart</title>
</head>
<body>
    <form id="form1" runat="server">
        <h2>Shopping Cart</h2>
        <asp:GridView ID="gvCart" runat="server"
OnRowCommand="gvCart_RowCommand">
            <Columns>
                <asp:BoundField DataField="ProductID" HeaderText="Product
ID" />
                <asp:BoundField DataField="ProductName" HeaderText="Product
Name" />
                <asp:BoundField DataField="Price" HeaderText="Price" />
                <asp:TemplateField>
                    <ItemTemplate>
                        <asp:Button ID="btnDelete" runat="server"
Text="Delete" CommandArgument='<%# Eval("ProductID") %>'
CommandName="DeleteItem" />
                    </ItemTemplate>
                </asp:TemplateField>
            </Columns>
        </asp:GridView>
        <asp:Label ID="lblTotal" runat="server" Text="Total: " />
    </form>
</body>
</html>
```

Code-behind (Cart.aspx.cs):

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace ShoppingCartApp
{
    public partial class Cart : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (Session["Cart"] != null)
            {
                List<Product> cart = (List<Product>)Session["Cart"];
                gvCart.DataSource = cart;
                gvCart.DataBind();
```

```
                // Calculate total price
                decimal total = cart.Sum(item => item.Price);
                lblTotal.Text = "Total: " + total.ToString("C");
            }
        }

        protected void gvCart_RowCommand(object sender,
System.Web.UI.WebControls.GridViewCommandEventArgs e)
        {
            if (e.CommandName == "DeleteItem")
            {
                int productId = Convert.ToInt32(e.CommandArgument);
                RemoveFromCart(productId);
            }
        }

        private void RemoveFromCart(int productId)
        {
            List<Product> cart = (List<Product>)Session["Cart"];
            Product itemToRemove = cart.FirstOrDefault(item =>
item.ProductID == productId);

            if (itemToRemove != null)
            {
                cart.Remove(itemToRemove);
                Session["Cart"] = cart;
                Response.Redirect("Cart.aspx"); // Refresh the page
            }
        }
    }
}
```

LAB 9: Entity Framework – Code First Approach

Objectives

- Learn how to implement CRUD operations (Create, Retrieve, Update, and Delete) using Entity Framework.
- Understand how to define models using the Code First approach.
- Use DbContext to manage the database and perform operations.
- Implement functionality to add, retrieve, update, and delete records in the database.

Theoretical Description

Entity Framework (EF)

- **Entity Framework** is an Object-Relational Mapper (ORM) that allows developers to interact with databases using object-oriented concepts. It automates the process of CRUD operations (Create, Retrieve, Update, Delete) by mapping database tables to classes in code.
- In the **Code-First** approach, the database schema is generated based on the models (classes) defined in the application. This means that the classes are created first in the code, and then Entity Framework generates the database structure accordingly.

Code-First Approach

- The **Code-First** approach allows developers to define their data models (classes) first and then use Entity Framework to automatically generate the database based on the model. In this approach:
    - **Models (classes)** are created first in the code.
    - Entity Framework uses these models to create the database schema.
    - The class properties correspond to table columns in the database.
    - You can then use **DbContext** to perform CRUD operations on the database.

Lab Task

- Create a new ASP.NET Web Application and add Entity Framework to the project.
- Define model classes for the data that will be stored in the database. For example, create a model class Product with properties like ProductID, ProductName, and Price.
- Create a DbContext class to manage the connection to the database and map the model classes to database tables.
- Perform CRUD operations
- Use Entity Framework's Migrations to generate and update the database schema based on the model classes.

Solution

1. **Step 1: Define Model Classes**
    - Create a model class for the data entity, for example, `Product`.

```
public class Product
{
    public int ProductID { get; set; }
    public string ProductName { get; set; }
    public decimal Price { get; set; }
}
```

2. **Step 2: Define DbContext Class**
    - Create a `DbContext` class to manage the connection and perform CRUD operations.

```
using System.Data.Entity;

public class ApplicationDbContext : DbContext
{
    public DbSet<Product> Products { get; set; }

    // Constructor: Set the connection string name
    public ApplicationDbContext() : base("name=YourConnectionString")
    {
    }
}
```

Note: Replace `"YourConnectionString"` with the actual connection string from `web.config` or `app.config`.

3. **Step 3: Configure Database and Migrations**
    - Open **Package Manager Console** and run the following commands to enable migrations and create the database:
        1. `Enable-Migrations` – This enables migrations for your project.
        2. `Add-Migration InitialCreate` – This generates the migration files based on your model.
        3. `Update-Database` – This applies the migration and creates the database.
4. **Step 4: CRUD Operations**

Add a New Product (Create Operation)

```
using (var context = new ApplicationDbContext())
{
    var product = new Product
    {
        ProductName = "Laptop",
        Price = 1200.00m
    };
    context.Products.Add(product);
    context.SaveChanges();
}
```

Retrieve Products (Read Operation)

```
using (var context = new ApplicationDbContext())
{
    var products = context.Products.ToList();
    foreach (var product in products)
    {
        Console.WriteLine($"ID: {product.ProductID}, Name:
{product.ProductName}, Price: {product.Price}");
    }
}
```

Update a Product (Update Operation)

```
using (var context = new ApplicationDbContext())
{
    var product = context.Products.FirstOrDefault(p => p.ProductID == 1);
    if (product != null)
    {
        product.Price = 1300.00m; // Update the price
        context.SaveChanges();
    }
}
```

Delete a Product (Delete Operation)

```
using (var context = new ApplicationDbContext())
{
    var product = context.Products.FirstOrDefault(p => p.ProductID == 1);
    if (product != null)
    {
        context.Products.Remove(product);
        context.SaveChanges();
    }
}
```

LAB 10: Threading and Synchronization in .NET

Objectives

- Learn how to create and manage threads in a .NET application.
- Understand synchronization techniques to prevent race conditions.
- Implement custom listeners to monitor thread activities.

Theoretical Description

Threading in .NET

- Threading allows you to run multiple operations simultaneously in a program, improving performance, especially for tasks like I/O operations or background processing.
- In .NET, threads can be created using the `Thread` class, which allows you to execute methods concurrently.

Synchronization

- Synchronization ensures that shared resources (e.g., variables, collections) are accessed by one thread at a time, preventing race conditions.
- Techniques like `lock`, `Mutex`, and `Monitor` are used to synchronize threads in .NET.

Listeners

- Listeners are used to monitor and log activities in a program. They are useful for debugging, auditing, and tracking the status of threads or events in the application.
- Custom listeners can be implemented to suit specific logging needs, like tracking thread states or handling thread exceptions.

Lab Task

- Create multiple threads to simulate simultaneous operations.
- Implement synchronization using lock or Monitor to ensure thread safety while accessing shared resources.
- Create a custom listener that logs the thread status and actions, such as thread start, completion, or exceptions.
- Execute the application and observe how the threads interact and how the listener captures thread details.

Solution

1. **Step 1: Create and Start Threads**

```
using System;
using System.Threading;

class Program
{
    static void Main(string[] args)
    {
        Thread thread1 = new Thread(new ThreadStart(DoWork));
        Thread thread2 = new Thread(new ThreadStart(DoWork));

        thread1.Start();
        thread2.Start();
    }

    static void DoWork()
    {
        Console.WriteLine($"Thread {Thread.CurrentThread.ManagedThreadId}
started.");
        // Simulate some work
        Thread.Sleep(2000);
        Console.WriteLine($"Thread {Thread.CurrentThread.ManagedThreadId}
completed.");
    }
}
```

2. **Step 2: Implement Synchronization**

```
using System;
using System.Threading;

class Program
{
    static object lockObj = new object();

    static void Main(string[] args)
    {
        Thread thread1 = new Thread(new ThreadStart(DoWork));
        Thread thread2 = new Thread(new ThreadStart(DoWork));

        thread1.Start();
        thread2.Start();
    }

    static void DoWork()
    {
        lock (lockObj)
        {
            Console.WriteLine($"Thread
{Thread.CurrentThread.ManagedThreadId} started.");
            // Simulate some work
            Thread.Sleep(2000);
            Console.WriteLine($"Thread
{Thread.CurrentThread.ManagedThreadId} completed.");
        }
    }
}
```

3. **Step 3: Create Custom Listener**

```
using System;
using System.Threading;

class CustomListener
{
    public void LogThreadStart(int threadId)
    {
        Console.WriteLine($"Thread {threadId} started.");
    }

    public void LogThreadCompletion(int threadId)
    {
        Console.WriteLine($"Thread {threadId} completed.");
    }
}

class Program
{
    static CustomListener listener = new CustomListener();

    static void Main(string[] args)
    {
        Thread thread1 = new Thread(new ThreadStart(DoWork));
        Thread thread2 = new Thread(new ThreadStart(DoWork));

        thread1.Start();
        thread2.Start();
    }

    static void DoWork()
    {
        listener.LogThreadStart(Thread.CurrentThread.ManagedThreadId);
        Thread.Sleep(2000);
        listener.LogThreadCompletion(Thread.CurrentThread.ManagedThreadId);
    }
}
```

LAB 11: Implementing MVC in .NET Web Applications

Objectives

- Learn the structure and components of the MVC (Model-View-Controller) pattern in .NET.
- Understand the role of each component (Model, View, Controller) in an MVC application.
- Develop a simple MVC application using Web Forms to demonstrate the integration of these components.

Theoretical Description

Model

- The model represents the data and business logic of the application. It is responsible for retrieving, storing, and manipulating data. The model is independent of the user interface, meaning it can be used across different types of views.

View

- The view is responsible for displaying the data provided by the model. It is the user interface of the application, where users interact with the application. In Web Forms, views are typically represented by `.aspx` files.

Controller

- The controller acts as an intermediary between the model and the view. It receives input from the user via the view, processes it (using the model), and updates the view accordingly. In MVC, the controller receives requests from the user, executes business logic, and updates the view with the results.

MVC Flow

- The user interacts with the view (UI), the controller handles the logic, and the model provides the data. The controller then updates the view, completing the flow.

Lab Task

- Create a Model class to hold data (for example, employee information).
- Create a View (Web Form) to display the data to the user.
- Create a Controller class to manage the logic and interact with the model and view.
- Connect everything in the Web Form to utilize the MVC pattern and display results.

Solution

1. **Step 1: Create the Model (Employee Model)**

```
// EmployeeModel.cs
public class EmployeeModel
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Position { get; set; }
    public double Salary { get; set; }
}
```

2. **Step 2: Create the Controller (EmployeeController)**

```
// EmployeeController.cs
using System;
using System.Collections.Generic;

public class EmployeeController
{
    // Create a list of employees to simulate data fetching
    public List<EmployeeModel> GetEmployees()
    {
        List<EmployeeModel> employees = new List<EmployeeModel>
        {
            new EmployeeModel { Id = 1, Name = "Ali", Position = "Manager",
Salary = 50000 },
            new EmployeeModel { Id = 2, Name = "Sarah", Position =
"Developer", Salary = 40000 },
            new EmployeeModel { Id = 3, Name = "Ahmed", Position =
"Designer", Salary = 35000 }
        };
        return employees;
    }
}
```

3. **Step 3: Create the View (Web Form to Display Employees)**

```
<!-- EmployeeView.aspx -->
<%@ Page Language="C#" AutoEventWireup="true"
CodeBehind="EmployeeView.aspx.cs" Inherits="WebApplication.EmployeeView" %>
<!DOCTYPE html>
<html>
<head>
    <title>Employee List</title>
</head>
<body>
    <h2>Employee List</h2>
    <table border="1">
        <thead>
            <tr>
                <th>ID</th>
                <th>Name</th>
                <th>Position</th>
                <th>Salary</th>
```

```
            </tr>
        </thead>
        <tbody>
            <% foreach(var employee in EmployeeList) { %>
                <tr>
                    <td><%= employee.Id %></td>
                    <td><%= employee.Name %></td>
                    <td><%= employee.Position %></td>
                    <td><%= employee.Salary %></td>
                </tr>
            <% } %>
        </tbody>
    </table>
</body>
</html>
```

4. **Step 4: Code-Behind for the View (Fetching Data in the Page)**

```
// EmployeeView.aspx.cs
using System;
using System.Collections.Generic;

public partial class EmployeeView : System.Web.UI.Page
{
    protected List<EmployeeModel> EmployeeList;

    protected void Page_Load(object sender, EventArgs e)
    {
        // Create an instance of the controller and fetch data
        EmployeeController controller = new EmployeeController();
        EmployeeList = controller.GetEmployees();
    }
}
```

LAB 12: LINQ to Objects/SQL/ DataSet/XML/JSON

Objectives

- Learn how to use LINQ (Language Integrated Query) to work with different data sources.
- Understand the concept of LINQ to Objects, LINQ to SQL, LINQ to DataSet, and LINQ to XML.
- Integrate JSON data and work with XML and JSON-based data in .NET.

Theoretical Description

1. **LINQ to Objects**
   - LINQ to Objects allows you to query in-memory collections like arrays, lists, or other data structures directly in C#. It provides a powerful syntax to filter, sort, and project data in a more readable and efficient manner than traditional looping constructs.

   Example: You can use LINQ to query an array of integers or a list of employees.

2. **LINQ to SQL**
   - LINQ to SQL is used to query SQL Server databases. It allows you to write queries directly in C# without having to write SQL. The queries are translated into SQL by the LINQ provider.

   Example: You can retrieve records from a database table using LINQ syntax and map them to a collection of objects.

3. **LINQ to DataSet**
   - LINQ to DataSet allows you to perform LINQ queries on DataSets (in-memory data structures used in ADO.NET). This is particularly useful when working with data disconnected from the database.

   Example: Querying a DataTable within a DataSet to filter or aggregate data.

4. **LINQ to XML**
   - LINQ to XML provides an easy way to work with XML data. You can load, query, and manipulate XML documents using LINQ in C#.

   Example: You can read and query XML data, such as filtering nodes or transforming XML content.

5. **JSON Integration**
   - JSON is widely used for exchanging data between applications. In .NET, you can use LINQ to query JSON data as well, often using libraries like `Newtonsoft.Json` (JSON.NET).

Example: Querying a JSON string or file to retrieve specific properties.

Lab Task

- LINQ to Objects: Create an array of employee objects and query the array using LINQ to filter and display employees based on a specific condition (e.g., salary).
- LINQ to SQL: Query a SQL Server database using LINQ to SQL to retrieve employee data.
- LINQ to DataSet: Create a DataSet with some sample data and use LINQ to query and filter the data.
- LINQ to XML: Load an XML document, query it using LINQ to find specific nodes or attributes.
- JSON Integration: Parse a JSON string and use LINQ to filter or manipulate the data.

Solution

1. **LINQ to Objects Example (Employee Array)**

```
using System;
using System.Linq;

class Program
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public double Salary { get; set; }
    }

    static void Main()
    {
        Employee[] employees = new Employee[]
        {
            new Employee { Id = 1, Name = "Ali", Salary = 50000 },
            new Employee { Id = 2, Name = "Sara", Salary = 60000 },
            new Employee { Id = 3, Name = "Ahmed", Salary = 45000 }
        };

        var highSalaryEmployees = from emp in employees
                                  where emp.Salary > 50000
                                  select emp;

        foreach (var emp in highSalaryEmployees)
        {
            Console.WriteLine($"ID: {emp.Id}, Name: {emp.Name}, Salary:
{emp.Salary}");
        }
```

```
    }
}
```

2. **LINQ to SQL Example**

```
using System;
using System.Linq;
using System.Data.Linq; // Include the required namespaces for LINQ to SQL

public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public double Salary { get; set; }
}

class Program
{
    static void Main()
    {
        // Assuming you have a database context named EmployeeDataContext
        using (var context = new EmployeeDataContext())
        {
            var employees = from emp in context.Employees
                            where emp.Salary > 50000
                            select emp;

            foreach (var emp in employees)
            {
                Console.WriteLine($"ID: {emp.Id}, Name: {emp.Name}, Salary:
{emp.Salary}");
            }
        }
    }
}
```

3. **LINQ to DataSet Example**

```
using System;
using System.Data;
using System.Linq;

class Program
{
    static void Main()
    {
        DataSet ds = new DataSet();
        DataTable dt = new DataTable("Employees");
        dt.Columns.Add("Id", typeof(int));
        dt.Columns.Add("Name", typeof(string));
        dt.Columns.Add("Salary", typeof(double));

        // Add some sample data
        dt.Rows.Add(1, "Ali", 50000);
        dt.Rows.Add(2, "Sara", 60000);
        dt.Rows.Add(3, "Ahmed", 45000);
```

```
        ds.Tables.Add(dt);

        var highSalaryEmployees = from row in
ds.Tables["Employees"].AsEnumerable()
                                  where row.Field<double>("Salary") > 50000
                                  select row;

        foreach (var row in highSalaryEmployees)
        {
            Console.WriteLine($"ID: {row["Id"]}, Name: {row["Name"]},
Salary: {row["Salary"]}");
        }
    }
}
```

4. **LINQ to XML Example**

```
using System;
using System.Linq;
using System.Xml.Linq;

class Program
{
    static void Main()
    {
        string xmlData = @"
        <Employees>
            <Employee>
                <Id>1</Id>
                <Name>Ali</Name>
                <Salary>50000</Salary>
            </Employee>
            <Employee>
                <Id>2</Id>
                <Name>Sara</Name>
                <Salary>60000</Salary>
            </Employee>
        </Employees>";

        XDocument doc = XDocument.Parse(xmlData);

        var highSalaryEmployees = from emp in doc.Descendants("Employee")
                                  where (double)emp.Element("Salary") >
50000
                                  select new
                                  {
                                      Id = emp.Element("Id").Value,
                                      Name = emp.Element("Name").Value,
                                      Salary = emp.Element("Salary").Value
                                  };

        foreach (var emp in highSalaryEmployees)
        {
            Console.WriteLine($"ID: {emp.Id}, Name: {emp.Name}, Salary:
{emp.Salary}");
        }
    }
}
```

5. **JSON Integration Example**

```
using System;
using System.Linq;
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;

class Program
{
    static void Main()
    {
        string jsonData = @"
        [
            { 'Id': 1, 'Name': 'Ali', 'Salary': 50000 },
            { 'Id': 2, 'Name': 'Sara', 'Salary': 60000 },
            { 'Id': 3, 'Name': 'Ahmed', 'Salary': 45000 }
        ]";

        JArray employees = JArray.Parse(jsonData);

        var highSalaryEmployees = from emp in employees
                                  where (double)emp["Salary"] > 50000
                                  select emp;

        foreach (var emp in highSalaryEmployees)
        {
            Console.WriteLine($"ID: {emp["Id"]}, Name: {emp["Name"]},
Salary: {emp["Salary"]}");
        }
    }
}
```

LAB 13: Introduction to .NET Core, ASP.NET Core, and Entity Framework Core

Objectives

- Understand the fundamentals of .NET Core.
- Learn the basics of ASP.NET Core and how it facilitates web application development.
- Explore the Code-First approach using Entity Framework Core.
- Get an overview of the basic structure of a web application in ASP.NET Core.

Theoretical Description

.NET Core Overview

- **.NET Core** is a cross-platform, open-source framework for building modern applications. It allows you to build applications that run on Windows, Linux, and macOS. It's lightweight, modular, and optimized for performance.

ASP.NET Core for Web Development

- **ASP.NET Core** is a framework used to build modern web applications and APIs. It's designed to be fast, lightweight, and secure. ASP.NET Core is built on **.NET Core**, making it cross-platform and highly scalable.

Entity Framework Core (EF Core)

- **Entity Framework Core** is an ORM (Object-Relational Mapper) for .NET Core. It simplifies database interactions by allowing developers to work with databases using C# objects. With **Code-First**, you define C# models first, and EF Core automatically creates the database from these models.

Lab Task

- Create a new ASP.NET Core Web Application in Visual Studio.
- Define a simple model class (e.g., Product) with properties like Id, Name, and Price.
- Set up EF Core to connect to a local database (SQL Server or SQLite).
- Run the application to see the basic structure and understand how the project is organized.

Solution

1. **Step 1: Create a Model Class (Product.cs)**

```
namespace MyApp.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

2. **Step 2: Set up DbContext (ApplicationDbContext.cs)**

```
using Microsoft.EntityFrameworkCore;

namespace MyApp.Models
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
            : base(options)
        {
        }

        public DbSet<Product> Products { get; set; }
    }
}
```

3. **Step 3: Configure EF Core in Startup.cs**

```
using Microsoft.EntityFrameworkCore;
using MyApp.Models;

public class Startup
{
    public void ConfigureServices(IServiceCollection services)
    {
        services.AddDbContext<ApplicationDbContext>(options =>
            options.UseSqlServer("Your_Connection_String_Here"));

        services.AddControllersWithViews();
    }
}
```

4. **Step 4: Create a Controller to Display Products (ProductController.cs)**

```
using Microsoft.AspNetCore.Mvc;
using MyApp.Models;

namespace MyApp.Controllers
{
    public class ProductController : Controller
    {
        private readonly ApplicationDbContext _context;
```

```
        public ProductController(ApplicationDbContext context)
        {
            _context = context;
        }

        public IActionResult Index()
        {
            var products = _context.Products.ToList();
            return View(products);
        }
    }
}
```

LAB 14: ASP.NET Core Web APIs

Objectives

- Understand the basics of ASP.NET Core Web APIs.
- Learn how ASP.NET Core Web APIs can be utilized for building cross-platform applications.
- Implement a simple ASP.NET Core Web API to demonstrate the principles.

Theoretical Description

ASP.NET Core Web APIs Overview

ASP.NET Core is a powerful framework for building web applications and APIs. A **Web API** (Application Programming Interface) allows different software systems to communicate with each other over the web. In **ASP.NET Core**, Web APIs are built using HTTP requests, making them ideal for communication between various platforms, such as web, mobile, and desktop applications.

Cross-Platform Applicability of ASP.NET Core Web APIs

One of the key features of **ASP.NET Core** is its cross-platform nature. Unlike older versions of ASP.NET, which were primarily Windows-based, **ASP.NET Core** is designed to run on multiple operating systems like **Windows**, **Linux**, and **macOS**. This cross-platform capability allows developers to create APIs that can be consumed by clients on different platforms

Lab Task

- Create a simple ASP.NET Core Web API for managing a list of products.
- Implement basic CRUD operations (Create, Read, Update, Delete) for the Product model.
- Use Postman or any HTTP client to test the API.
- Test the Web API from a cross-platform application (e.g., using a mobile app or a simple HTML/JavaScript page).

Solution

1. **Step 1: Create a Model Class (Product.cs)**

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

2. **Step 2: Set up the Web API Controller (ProductController.cs)**

```
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;

namespace MyApp.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        private static List<Product> products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 1000 },
            new Product { Id = 2, Name = "Phone", Price = 500 }
        };

        // GET: api/product
        [HttpGet]
        public IEnumerable<Product> Get() => products;

        // POST: api/product
        [HttpPost]
        public IActionResult Post([FromBody] Product product)
        {
            products.Add(product);
            return Ok();
        }

        // PUT: api/product/5
        [HttpPut("{id}")]
        public IActionResult Put(int id, [FromBody] Product product)
        {
            var existingProduct = products.Find(p => p.Id == id);
            if (existingProduct != null)
            {
                existingProduct.Name = product.Name;
                existingProduct.Price = product.Price;
                return Ok();
            }
            return NotFound();
        }

        // DELETE: api/product/5
        [HttpDelete("{id}")]
        public IActionResult Delete(int id)
        {
            var product = products.Find(p => p.Id == id);
            if (product != null)
            {
                products.Remove(product);
                return Ok();
            }
            return NotFound();
        }
    }
}
```

3. **Step 3: Configure the Application (Startup.cs)**

```
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

namespace MyApp
{
    public class Startup
    {
        public void ConfigureServices(IServiceCollection services)
        {
            services.AddControllers();
        }

        public void Configure(IApplicationBuilder app, IWebHostEnvironment
env)
        {
            if (env.IsDevelopment())
            {
                app.UseDeveloperExceptionPage();
            }

            app.UseRouting();
            app.UseEndpoints(endpoints =>
            {
                endpoints.MapControllers();
            });
        }
    }
}
```

LAB 15: Project Evaluation

Objectives

This lab is based on project evaluation, intended to take viva of students regarding their project in Visual Programming

Lab Task

Students will present their projects and answer the questions asked in response.

Open Ended Lab (OEL)

Course: Visual Programming **Total Marks: 15**

OEL Title: Enhancing Database Operations with Stored Procedures in .NET Applications

OEL Level: 1

Objectives:

- Understand the purpose and benefits of using stored procedures in database-driven applications.
- Learn how to create, modify, and execute stored procedures in SQL Server.
- Integrate stored procedures into a .NET application for efficient database operations.

Background Information:

Stored procedures are precompiled SQL statements stored in a database. They improve performance by reducing the need to send multiple queries from the application to the database and enhance security by limiting direct SQL execution. In this lab, students will explore how to create and use stored procedures and integrate them into a .NET application using ADO.NET or Entity Framework.

Problem:

You are hired by a company to optimize the performance of their customer management system. The system allows users to view, add, update, and delete customer information. Currently, all operations are handled with inline SQL queries, leading to poor performance and security vulnerabilities.

Your task is to redesign the system to use stored procedures for database interactions. This will enhance performance and secure the database operations.

Requirements:

Create stored procedures for:

- Retrieving all customers.
- Adding a new customer.
- Updating customer details.
- Deleting a customer.

Database Integration:

- Integrate these stored procedures into a .NET application.
- Use ADO.NET or Entity Framework to execute the stored procedures.

UI Implementation:

- Develop a simple console or Windows Forms application that:
- Lists all customers.
- Allows the user to add, update, and delete customer records.

Error Handling:

- Implement proper error handling for database operations to handle scenarios like duplicate entries or invalid input.

Documentation:

- Provide comments in the code explaining the purpose of each stored procedure and its integration in the application.